Software Testing - Zusammenfassung Sommersemester

Fortgeschrittene Themen des Softwaretests (Technische Universität München)

Scan to open on Studocu

Downloaded by Wang Youran (euzil12138@gmail.com)
Studocu is not sponsored or endorsed by any college or university

Software Testing

Montag, 15. April 2024 12:10

Introduction:

- Goal: show intended functionality is correctly implemented and detect failures
- Test driven development: Implement tests before developing the code
- **Verification**: Building the right system
- **Validation**: Build the system right
- Failure: Actual not Intended behavior (Runtime)
- Error: System actual state is not intended state
- Fault: Mistake in the code
- White box testing: using controlflow, dataflow and stochastical profiles for testing
- Code coverage: messure the amount of code tested
- Oracel: known solution

Test Selection:

- Testing activities: Planning, organization, documentation, test case derivation, execution, monitoring, evaluation
- **Unit tests**: Testing one part of the Programm alone
- **Integration tests**: Combining some Units
- **System tests**: Test whole system
- Different tests: Black-Box, White-Box, Robustness, Performance, Regression (testing if change messed up the system)

Integration test:

- **Integrationfaults (can be blamed on multiple Componants)**: Interaction (Inferstructure) Fault, Design Fault (e.g Communication)
- Integrationtesting depends on a Specification or test what goes wrong in the past

All Integration faults:

- Unit , Coding, Data model, quantization
- Time, frequency, value range, format
- Data type, message synchronicity, communication, message ordering
- call ordering, schedule, information freshness, WCET vs. timeout

Integration levels:

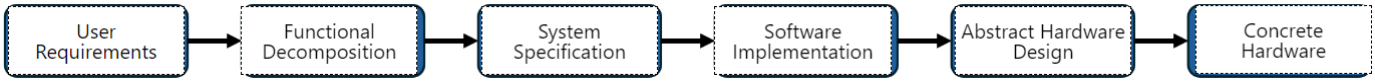
- Unit
- Horizontal Softwareintegration

studocu

Scan to open on Studocu
Downloaded by Wang Youran (euzil12138@gmail.com)

- Horizontal Softwareintegration
- Vertical Software/Hardware integration
- Software in the Loop
- Hardware in the Loop
- Horizontal Hardware Integration (Integration between different Servers/Clients)

Decomposing userrequierments:



Equivalence Partitioning:

- Group input data into blocks (e.g positiv numbers, negative numbers)
- Test of functionality or test a potential fault

Control Flow-Based Testing:

- **Control Flow Based Testing**: increase coverage
- Data flow based Testing: Assigment of values to variables needs to be tested
- **Statement Coverage**: Each line of code should have been executed at least once (Each node in the Graph)
- **Path Coverage**: Every path from the entry to the exit point is executed once (Each Edge in the Graph)
- Path testing not practical because of loops
- **Branch Coverage**: every branch in the controll flow graph is evaluated once
- **Decision Coverage**: every case once evaluate to true and flase
- **Condition Coverage:** each literal have to be true and false once
- **Condition/Decision** Coverage: combines condition and decision Coverage
- **MCDC**: and decision coverage (for n variables n+1 testcases) **?????**
- **Multiple Condition Coverage:** All combinations of literals (2^n)
- **Boundary interior testing**: Find boundary value for example Max integer and test them

Coverage criteria:

- Selection criteria (before testing)
- Quality indicators (after testing)
- Stopping criteria (end testing)

Combinatorial Testing:

- Cobinatorial Testing: Assumption Failures are result of two combinations
- Only test the pairs

Random Testing:

General:

- Inputs can be selected uniformly or in a probability distibution

studocu
Scan to open on Studocu
Downloaded by Wang Youran (euzil12138@gmail.com)

- Instead of ranom input create State maschine and test all states
- **Random adaptiv testing**: Assumption failuers appear in a cluster
- Testing for Exception or also adding Sanatizer as oracel

Fuzzing:

- **Fuzzing**: throw a lot of semi valide input on the code (most time security testing)
- **Black box fuzzing**: Fuzzing without the knowledge of the internal behavior
- **Gray box fuzzing:** using some informations from the system for the Mutations
- **White box fuzzing:** detect program paths and fuzz each path
- **Seed**: Data to start testing for semi-random input (fuzzing sensitiv for seed)
- Inferred model: No knowledge of the program's input format
- Dictionary-based Mutations: changing words to different words
- Crossover: multiple testcases get mixed
- Deduplication: which Failuers lead to the same Fault?

Techniques:

- Random testing
- Model based testing
- Mutation based testing

Defect hypothesis:

- Create input that the programmer had not thought about
- Interesting inputs are semi valide
- Statements like memcpy are problematic

Random testing in Practice:

- **Property-based testing**: to solve the oracel problem use some information what should come out (general things: do not crash)

Random testing advantages:

- Saves time
- Unbaiesed
- Good for security

Disadvantages:

- A lot of redundant or unrealistic
- No Oracel
- Often only capable of finding basic bugs
- Fault localization is more difficualt

Analysis Random Testing vs Partition Based Testing:

Model of Weyuker & Jeng:

- Input domain D of size d of which m points produce incorrect output
- Failure rate $\theta_i = \frac{m_i}{}$

studocu

Scan to open on Studocu
Downloaded by Wang Youran (euzil12138@gmail.com)



Failure rate $\theta_i = \frac{}{|D_i|}$

- Probability that **random testing** detecting at least one failure: $P_r = 1 - (1-\theta)^n$
- Probability for **partion testing** detecting at least one failure: $P_p = 1 - \prod_{1 \le i \le k}(1-\theta_i)^{n_i}$
- All failure are part of one small block then partition based testing is better
- General pation based testing does not increase the likelihood of finding an failure
- **Assumtion of this model:** Failure rate is fix and tester has to know the failure rate
- If all blocks are the same size partion based can not be worse than random testing

Gutjahr's Model:

- Failure rates can be calculated on statistical values
- Failure rates become random variables then expected value can be calculated
- If **expected failure rates** are identical than partition based testing will never be worse then random testing

Adaptive Partion Testing:

- Chose one Block of the partition on success reduce the likelihood of choosing the Block

Testing Concurrent Programs:

- Problems: Deadlock, Lifelock, Starvation, Suspension, Order Violation, Priority Inversion, Atomicity Violation
- **Blocking synchronization**: locking the ressource accessed
- **non Blocking synchronization**: Rejecting other thread if trys to access the ressource

Testing strategies:

- Force some execution order
- Instead using time you can synchronize the Programm with a Logical Clock
- Or Specify desired order of events (Framework IMUnit)

Testing Cyber Physical Systems and Machine Learned Models:

Testing Cyber Physical System:

- Problems: No useful requirements, no clear oracel, dynamic open context

X-in-the-Loop-testing:

- test setup on the simulation enviroment
- **Model-in-the-Loop**: Model of the system is run in the simulation environment
- **Software-in-the-Loop**: Actual software is used; sometimes on the dedicated computation hardware

studocu
Scan to open on Studocu
Downloaded by Wang Youran (euzil12138@gmail.com)

computation hardware
- **Hardware-in-the-Loop**: Actual hardware is used, e.g. mechanical steering parts

Search-Based Test Case Generation:
- Input -> Search Based Optimization -> Output
- Search Based Optimization: Simulating the system -> Find out the quality of testcases -> Optimize
- Good test case leads to the worst behavior
- **search space**: Environment in which the system is tested
- **fitness function**: evaluates a test case

Genetic Algorithms:
- **Structure**: Gene (One dimension in search), Individual: (Gen for each dimension), Population (Individual test Cases)
- **Process**: Compute fitness for all population
- Convergates to local but not always global optimum

Test Goals Autonomos Driving:
- Do not use a heuristic that does not necessarily correlate with faulty system behavior
- Doing partition based testing
- **Step 1**: Define fitness function tampletes reasonable
- **Step 2**: Define fitness function template for good test cases
- **Step 3**: Combine instantiated templates to yield good test cases
- Search goal: Violation of the safe operating envelope
- Search for test cases in a specific threshold

<u>Testing Machine Learned Models:</u>

Test Level:
- **Component level testing**: test single machine learning model (training, validation)
- **System level tests**: Test the complete system (watchdog)

<u>Metamorphic testing:</u>
- **Metamorphic testing**: Create Metamorphic relations between input and output (e.g add fog to image result should be the same)
- Advantages: Cost effective, Works for Machine learning and Not Machine learning
- Disadvantages: Domain knowledge needed,

<u>Robustness testing:</u>
- Gini coficient: meassures how far of a test case is from the models output distribution

Symbolic execution:
1. Translate Neural network to imperatvie program
2. Detect most important pixel

studocu
Scan to open on Studocu
Downloaded by Wang Youran (euzil12138@gmail.com)

2. Detect most important pixel
3. Create adversarial image from the original image

Neuron Coverage Criteria:

- **Discretize Neuron Activation Range**: Activation of neuron will be considered in discret input space
- **Continouse Neuron Activation Range**: Activation of neuron will be considered in continuze input space
- Different metrics (e.g total numbers of Neuron activated, for each neuron partition output values in k sections)

1. Partition input space
2. select a seed image and apply transformation to trigger other neurons

Laten Space Coverage:

- Input Space: Data points
- Laten Space: Create Clusters and chose the points which are not part of a cluster

Test Ending Criteria, Regression Testing, Flaky Testing:

Test Ending Criteria:

- **Fault Injection**: Integrating faults to test if testcases detect the fault
- Mutation Score: Divide number of failed tests by number of mutation made
- Mutant detection correlate with fault detection

Regression Testing:

- **Regression testing**: run tests if changes made on the code caused failure
- **Regression test prioritization**: check if testcase is important for the changes made and prioritize them
- Test case priorization: value based optimazation (e.g coverage), cost based optimazation (e.g execution time)
- Test suite minimization: remove test cases which are not important
- Test case selection: Test only test cases affected by changes

Flakey Testing:

- Test fails or passes on two different occassions even the code remains the same

Model Based Testing:

Abstraktion:

- **Model under test** and **Enviroment** can have different kind of abstration
- Purpose: Insight into System, Specification, Code generation
- Second kind of abstraction: Simplification with loss of infomation
- **encapsulation**: parts of the system will not be visible for the user
- **omission**: details are not known

studocu

Scan to open on Studocu
Downloaded by Wang Youran (euzil12138@gmail.com)

Scenarios:

- Tests and Code generated from 1 Model: does not work
- Test and Code generated from 2 Models: very expensive
- Model only for Testcases: typical approache

Selection Criteria:

- Functional criteria: Requirements testing (not defect based)
- Structural criteria: Dataflow, Controlflow, etc.
- Stochastik criteria: Uniformal distribution
- Defectbased criteria:

Generation Test Cases:

- Search problem: find a state or an output or parameters
- Problem: what is a good testcase

Testing Object Oriented Software:

Statebased testing:

- **Statebased testing**: Class defines state variables and functions methods on them
- Interprete code as extended finite state machine and check fault models

Impact on testing:

- State dependent behavior
- Encapsulation
- Inheritance
- Polymorphism
- Abstract and generic classes

Control faults:

- Missing or incorrect transtion of events
- Missing or incorrect action
- Extra, missing or corrupt state
- Sneak path or illegal message failure

Flattening:

- **fattening**: Flattend class makes all inherited features explicit
- Extension: Automatic inclusion of superclass featrues
- Overriding: Same method in super and subclass
- Specialization: Addition of class features

Faults:

- Naughty children: subclass leaves object in state that is illegal in superclass
- Worm holes: subclasses not consistent with superclass invariant

studocu
Scan to open on Studocu
Downloaded by Wang Youran (euzil12138@gmail.com)